

## 201 Malware Operasyonu — The Malware Hunter

Ad Soyad: Berat Can Satır

Eğitim: Siber Güvenlik Mühendisliği Eğitimi -2026

Görev: 201 Malware Operasyonu

Tarih: 05 Mayıs 2026

### FAZ 1 — MİMARİ VE TEMELLER

#### 1.1 Analiz Türleri: Durağan ve Hareketli

##### Statik Analiz Nedir?

Statik analiz, bir dosyayı hiç çalıştırmadan inceleme sürecidir. Burada amaç, dosyanın davranışını tahmin etmek için içindeki ipuçlarını toplamaktır. Bu süreçte şunları yaparız: dosyanın hash değerini hesaplayarak kimliğini doğrularız, PE başlık bilgilerini okuruz, içindeki okunabilir metinleri (strings) tarayarak IP adresi, URL ya da registry yolu gibi kritik kanıtlar ararız, hangi DLL kütüphanelerini import ettiğine bakarız.

Kullandığımız başlıca araçlar: PESTudio, CFF Explorer, FLOSS, capa.

##### Dinamik Analiz Nedir?

Dinamik analiz, zararlıyı izole bir ortamda çalıştırıp davranışlarını gerçek zamanlı izleme sürecidir. Hangi dosyaları oluşturduğunu, hangi registry anahtarlarına dokunduğunu, hangi dış IP'lere bağlantı kurduğunu ve hangi processleri başlattığını canlı olarak gözlemliyoruz.

Kullandığımız başlıca araçlar: ProcMon, Process Hacker, Wireshark, Regshot.

#### Neden Önce Statik, Sonra Dinamik?

Bu sıranın iki temel nedeni var.

Birincisi güvenlik: Dosyanın ne kadar tehlikeli olduğunu bilmeden doğrudan çalıştırmak, kontrol edemeyeceğimiz sonuçlara yol açabilir. Statik analiz bize ilk risk değerlendirmesini sağlar ve sandbox kurulumunu buna göre sıkılaştırmamıza imkân tanır.

İkincisi strateji: Statik analizde tespit ettiğimiz API çağrıları, string'ler ve import listesi, dinamik analizde neye bakacağımızı söyler. Örneğin PESTudio'da WSASStartup veya InternetOpenA import'larını görürsek, dinamik analizde ağ trafiğine odaklanmamız

gerektiğini önceden biliriz. Rehbersiz dinamik analiz, binlerce ProcMon eventi içinde kaybolmak demektir.

## 1.2 Güvenli Alan: Kafes (Sandbox)

Sandbox, gerçek sistemden tamamen izole edilmiş kontrollü bir çalışma ortamıdır. Zararlıyı kendi ana makinemizde çalıştırmak yerine neden sanal bir makine kullanırız?

İzolasyon: Sanal makine, host işletim sisteminden ayrı bir ağ ve dosya sistemi katmanında çalışır. Zararlı, VM dışına çıkamaz. Gerçek dosyalarımıza, diğer makinelere ve internete erişimi tamamen kesilebilir.

Geri Alınabilirlik (Snapshot): VM'in temiz halinin anlık görüntüsünü alırız. Analiz bittiğinde tek tıkla o temiz hale döneriz. Gerçek bir sistemde bu mümkün değildir.

Gözlemlenebilirlik: VM içine ProcMon, Wireshark gibi izleme araçları kurulur. Zararlının her hareketi kayıt altına alınır. Host sistemde bu kadar kapsamlı izleme pratikte yapılamaz.

Ağ Kontrolü: INetSim veya FakeNet gibi araçlarla C&C sunucusu taklit edilir. Zararlı gerçekten internete çıkamaz ama bağlandığını sanır. Bu sayede hangi domain veya IP'ye ulaşmaya çalıştığını güvenle öğreniriz.

## 1.3 Tersine Mühendislik (Reverse Engineering): Kodu Geriye Sarmak

Tersine mühendislik, derlenmiş bir ikili dosyanın (binary) kaynak kodunu elde etmeden davranışını ve mantığını anlamaya çalışma sürecidir. Bir geliştirici C veya C++ kodunu derleyince ortaya çıkan .exe dosyası artık makinenin anlayacağı assembly ve makine kodudur; insan okuyabilir kaynak kod yoktur.

IDA Pro veya Ghidra gibi araçlar bu binary'yi alıp bize assembly listesi ya da pseudo-C kodu olarak sunar. Bu kodda şu sorulara cevap arayabiliriz:

Hangi Windows API fonksiyonlarını çağırıyor? CreateFile, RegSetValueEx, WSASend gibi çağrılar zararlının ne yapmayı planladığını açıkça gösterir.

Koşullu dallanmalar ve döngüler hangi değerleri kontrol ediyor? VM tespit kodu (anti-VM), debug tespiti (IsDebuggerPresent), şifreleme döngüleri bunların başında gelir.

Hangi string'ler bellekte inşa ediliyor? Obfuscated string'ler çoğunlukla runtime'da XOR ile açılır; assembly'de bu döngüyü görebiliriz.

Kısacası tersine mühendislik, "Bu .exe çalıştırılırsa ne olur?" sorusunu güvenli ve metodolojik biçimde cevaplar. Antivirüsün tanıyamadığı yeni zararlılar için tek güvenilir analiz yoludur.

## FAZ 2 — SAHA OPERASYONU (Boss Fights)

### Cephe 1: Dosyayı Kesip Bıçmek -Basic Static Analysis

#### Ortam Hazırlığı

Oda başlatıldığında FLARE VM içeren bir Windows sanal makinesi açıldı.

Desktop\mal\ dizininde 1'den 6'ya kadar numaralandırılmış zararlı örnekler bulunmaktaydı.

#### Hash Tespiti

Bir dosyanın hash değeri onun parmak izidir. Aynı dosyanın hash'i her zaman aynıdır; tek bir byte değiştiğinde hash tamamen farklılaşır. Hash hesaplamak için PowerShell kullandım

```
Get-FileHash .\mal\1 -Algorithm MD5
```

```
Get-FileHash .\mal\1 -Algorithm SHA256
```

mal\3 ile aynı imphash değerine sahip dosya: mal\1 ssdeep fuzzy hash benzerlik oranı: %93

Bu yüksek benzerlik, her iki dosyanın büyük ihtimalle aynı kaynak koddan türetildiğine işaret ediyor. Bu tür karşılaştırmalar, aynı zararlı ailesine ait örnekleri kümelememizi sağlar.

#### String Analizi - Zararlının Niyetini Okumak

FLOSS aracı hem düz okunabilir string'leri hem de XOR ve stack-string gibi gizlenmiş string'leri ayıklar

```
floss.exe Desktop\mal\2 > Desktop\mal\output2.txt
```

```
floss.exe Desktop\mal\5 > Desktop\mal\output5.txt
```

```
floss.exe Desktop\mal\6 > Desktop\mal\output6.txt
```

"DbgView.exe" string'ini içeren örnek: mal\6

Bu string son derece anlamlı: DbgView (Debug View), geliştiricilerin ve analistlerin kullandığı bir Sysinternals aracıdır. Zararlının bu string'i içermesi, çalışmadan önce debug araçlarının açık olup olmadığını kontrol ettiğini gösteriyor. Bu klasik bir anti-debugging tekniğidir.

PEStudio ile PE Header Analizi

mal\4 örneğini PEStudio'ya sürükleyip Imports sekmesini açtım. Blacklist sütununda kırmızı işaretli kütüphane: Remote Procedure Call Runtime (rpcrt4.dll)

capa ile Kapasite Analizi

capa, binary'nin neler yapabileceğini MITRE ATT&CK çerçevesine göre eşleştirir:

capa.exe -v Desktop\mal\4 > Desktop\mal\capa4.txt

86 anti-VM eşleşmesi dikkat çekici. Bu kadar yüksek bir sayı, zararlının sanal makinede çalıştığını tespit etmeye çok büyük önem verdiğini ortaya koyuyor. Eğer VM tespit ederse davranışını değiştiriyor ya da tamamen duruyor.

Cephe 2: Canavarı Serbest Bırakmak Basic Dynamic Analysis

Ortam ve Güvenlik Önlemleri

Dinamik analize geçmeden önce sanal makinenin temiz halinin snapshot'ını aldım. Ağ bağdaştırıcısını Host-Only moduna getirerek dış internetten kestim. Ardından araçları sırayla başlattım: Desktop > Tools > Utilities > procmon.exe

ProcMon ile 1.exe Analizi — Ağ Bağlantıları

ProcMon başlatıldı, tüm mevcut eventler temizlendi. "Process Name is 1.exe" filtresi eklenerek gürültü azaltıldı. Ardından 1.exe çalıştırıldı.

[EKRAN GÖRÜNTÜSÜ 2.1 — ProcMon'da 1.exe'ye ait TCP Connect eventleri. İlk ağ bağlantısının kurulduğu URL görülmektedir.]

ProcMon'da Network eventlerine odaklandığımda 1.exe'nin belirli aralıklarla dışarıya bağlantı kurduğunu gözlemledim. Bu düzenli bağlantı örüntüsü, C&C iletişiminin tipik bir göstergesidir (beaconing davranışı).

[EKRAN GÖRÜNTÜSÜ 2.2 — 1.exe'nin C:\ dizininde oluşturduğu dosya. ProcMon File System eventleri filtrelenmiş halde görülmektedir.]

## Process Explorer ile Process Hollowing Tespiti

Odadaki Lew.exe örnekleri Process Explorer ile incelendi. Bir process bellek görüntüsü diskteki dosya ile eşleşmiyorsa Process Explorer o process'i farklı bir renkte gösterir.

Sorunun cevabı: N (Hayır) — Process bellekte ve diskte aynı değil. Bu, process hollowing saldırısının açık kanıtı.

Process hollowing şöyle çalışır: Zararlı önce meşru bir process başlatır, sonra o process'in bellek alanını boşaltıp kendi kodunu oraya enjekte eder. Dışarıdan bakıldığında meşru bir process çalışıyor gibi görünür.

## Regshot ile Registry Değişikliklerini İzlemek

3.exe çalıştırılmadan önce 1. Regshot alındı. Ardından 3.exe çalıştırıldı, kısa bir süre beklendi ve 2. Regshot alındı. "Compare" butonuyla iki snapshot arasındaki fark raporlandı.

Delta raporunda HKU\S-X-X-XX-...\Software\Microsoft\Windows\CurrentVersion\Run altında yeni bir değer eklendiği görüldü. Bu değer 3.exe'nin tam yolunu içeriyordu. Zararlı, kalıcılık sağlamak amacıyla kendini kullanıcı hive'ına kaydetmişti; bilgisayar her yeniden başlatıldığında otomatik çalışacaktı.

## Cephe 3: Malware Hunter MalBuster

Bu cephe, öğrendiğim tüm statik analiz tekniklerini dört gerçek zararlı örnek üzerinde birleştirmemi gerektirdi. Örnekler /home/ubuntu/Desktop/malbuster/ dizinindeydi.

### malbuster\_1 Analizi

Adım 1 — Mimari tespiti:

PE-bear aracında FILE\_HEADER bölümüne baktım. Machine alanı: 0x014C (Intel 386). OPTIONAL\_HEADER Magic: 0x010B. Bunlar dosyanın 32-bit bir uygulama olduğunu kesin biçimde ortaya koyuyor.

Adım 2 - MD5 Hash hesaplama:

md5sum malbuster\_1

Adım 3 - VirusTotal ile tehdit tespiti:

Hash değerini VirusTotal'a yapıştırdım. Yüksek sayıda antivirüs tarafından tespit edildi. Popular threat label: Emotet

Emotet, dünyanın en tehlikeli botnet altyapılarından biri olarak biliniyor. E-posta yoluyla yayılıyor, kurban sistemde hem kalıcılık sağlıyor hem de TrickBot gibi ikincil zararlıları indirip çalıştırıyor.

Adım 4 capa ile anti-VM analizi

capa malbuster\_1 | grep -i "anti-vm"

Anti-VM instruction sayısı: 3

Adım 5 - Kritik string tespiti

strings malbuster\_1 | grep -i "mozilla"

Bulunan kritik string: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1)

Bu string son derece önemli. Zararlı, C&C sunucusuna gönderdiği HTTP isteklerini eski bir Internet Explorer trafiği gibi göstererek ağ izleme sistemlerini atlatmaya çalışıyor. 2026 yılında hiçbir gerçek kullanıcı bu User-Agent'ı kullanmaz; ağ trafiğinde bu string'i görmek tek başına şüphe işaretidir.

malbuster\_2 Analizi

PEStudio'ya malbuster\_2 yüklendi. Imports sekmesinde \_CorExeMain fonksiyonu tespit edildi.

\_CorExeMain hangi DLL'den import ediliyor: mscoree.dll

Bu import, dosyanın bir .NET uygulaması olduğunu kesin biçimde ortaya koyuyor. mscoree.dll, Microsoft Common Object Runtime Execution Engine'in kütüphanesidir; .NET uygulamalarını başlatmaktan sorumludur.

VS\_VERSION\_INFO — Orijinal dosya adı:

PEStudio'nun Version Info sekmesinde OriginalFilename alanı: Nyan.exe

Avira detection (VirusTotal'dan): Zararlının imzası görüldü.

GodMode string'ini hangi binary içeriyor: malbuster\_2

strings malbuster\_2 | grep -i "god"

*malbuster\_3 ve malbuster\_4 Analizi*

malbuster\_3:

SHA256 hash alındı ve abuse.ch MalwareBazaar veritabanında arandı.

İmza: TrickBot

capa analizi kritik bir bulgu verdi:

capa malbuster\_3 | grep -i "keylog"

malbuster\_3, keylogger kapasitesine sahip. MITRE teknik ID: T1056 (Input Capture). Bu, zararlının kurban sistemde klavye girdilerini kaydedip saldırgana ilettiği anlamına geliyor.

malbuster\_4:

SHA256 hash abuse.ch'e sorgulandı. İmza: ZLoader

ZLoader, öncelikli olarak bankacılık kimlik bilgilerini çalmak için kullanılan bir zararlı. Web inject teknikleriyle tarayıcı oturumlarına müdahale ediyor.

ShellExecuteA hangi DLL'den import ediliyor

Cevap: SHELL32.DLL

DOS\_STUB mesajı:

PE-bear aracında DOS Stub bölümü açıldı.

Mesaj: !This Salfram cannot be run in DOS mode

Normal bir PE dosyasında bu mesaj "This program cannot be run in DOS mode" şeklindedir. "Salfram" kelimesi kasıtlı olarak yerleştirilmiş; bu tür değişiklikler zararlı yazılım geliştiricilerinin yazar imzası gibi kullanmasıyla ya da tespitten kaçınmak amacıyla yapılır.

MITRE Discovery tekniği: capa çıktısında T1083 (File and Directory Discovery) görüldü. ZLoader'ın kurban sistemdeki dosya yapısını keşfetmeye çalıştığı anlaşılıyor.

## **SONUÇ VE DEĞERLENDİRME**

Bu operasyonda statik analiz, dinamik analiz ve tehdit istihbaratı araçlarını bir arada kullanarak dört gerçek zararlı örneği başarıyla analiz ettim. Temel bulgular:

Bu operasyonun bana öğrettiklerini bir cümleyle özetleyecek olursam: antivirüs bir kapı bekçisidir, mühendis ise kapıyı nasıl aştıklarını anlayan kişidir. Statik analiz bize hedefli bir dinamik analiz stratejisi sağlar; assembly okuyabilmek hiçbir aracın tanıyamadığı yeni tehditleri çözümlemek için vazgeçilmezdir; kalıcılık mekanizmalarını bilmek ise bir enfeksiyonu tam anlamıyla temizlemek için şarttır.

*AnkaCORE Operasyon Merkezi* 🦅